API & Service-Level Continuous Testing

Day 5 · Module 5 · 2 Hours

SDET / DevOps Engineer

Intermediate

35% Theory / 65% Hands-on

TOOLS

REST Assured 6.0 (Java 17+)

Postman / Newman CLI

LEARNING OUTCOMES

What You Will Achieve Today

Implement API tests in CI pipelines

Write automated REST API tests using REST Assured (Java) and Postman/Newman, and wire them into your CI pipeline as first-class quality gates.

Validate service contracts and integrations

Apply contract testing, JSON schema validation, and response assertion patterns to catch integration defects before they reach staging or production.

Improve early defect detection at the service layer

Replace slow, brittle UI tests with fast, stable API checks. Understand where API tests sit in the test pyramid and when to use them vs unit or E2E tests.

API Testing in CI — Why and Where

📍 In the test pyramid: API/Integration tests sit between unit tests and E2E UI tests. They are faster and more stable than UI tests but catch integration defects that unit tests miss. Target ~30% of your suite.

What to Validate in API Tests

Status Codes

200 OK, 201 Created, 400 Bad Request, 401 Unauthorised, 404 Not Found, 500 Internal Server Error — test boundary conditions, not just happy path.

Response Body

Correct fields present, right data types, expected values using JSONPath assertions. Avoid asserting on volatile data (timestamps, IDs).

Response Headers

Content-Type: application/json, Cache-Control, CORS headers, rate-limit headers. Often overlooked but contractually important.

JSON Schema

Validate the shape of the response — field names, types, required fields — against a defined schema. Catches contract drift before consumers break.

REST Assured vs Postman/Newman

Dimension	REST Assured	Postman / Newman
Language	Java (DSL)	JavaScript / JSON collection
Best for	Java codebases, CI-native	GUI exploration + CI via Newman
Version control	Code in Git natively	Export JSON + commit
CI execution	mvn test / ./gradlew test	newman run collection.json
Assertions	Hamcrest / AssertJ	Postman test scripts (JS)
Java 17+ req	YES (v6.0.0+)	Node.js runtime

REST Assured 6.0 — Setup & First API Tests

pom.xml — REST Assured 6.0.0 dependencies

```
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>6.0.0</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>json-schema-validator</artifactId>
  <version>6.0.0</version>
  <scope>test</scope>
</dependency>
<!-- REST Assured 6.x requires Java 17+       -->
<!-- Add json-schema-validator for schema checks -->
```

Basic GET test — given / when / then DSL

```
// Static imports required for given/when/then DSL
import static io.restassured.RestAssured.*;
import static io.restassured.matcher
    .RestAssuredMatchers.*;
import static org.hamcrest.Matchers.*;

@Test
void getUser_returns200_withExpectedName() {
  given()
    .baseUri("https://api.example.com")
    .header("Accept", "application/json")
  .when()
    .get("/users/1")
  .then()
    .statusCode(200)
    .contentType(ContentType.JSON)
    .body("name", equalTo("Alice"))
    .body("email", notNullValue())
    .header("Content-Type",
            containsString("application/json"));
}
```

REST Assured Key Concepts

given / when / then DSL

Mirrors BDD syntax. given() sets up request spec (base URI, headers, body, auth). when() fires the HTTP method. then() asserts on the response. Highly readable — tests serve as documentation.

RequestSpecification — reuse base config

Extract common config (base URL, auth header, content type) into a shared RequestSpecification. Avoids duplication across all test methods; change once, applies everywhere.

JSONPath assertions

body("user.name", equalTo("Alice")) navigates nested JSON. body("users.size()", greaterThan(0)) checks array length. body("ids", hasItems(1, 2, 3)) validates list membership.

JSON Schema Validation

matchesJsonSchemaInClasspath("schemas/user-schema.json") validates response structure against a JSON Schema file — ensures contract compliance beyond individual field assertions.

v6.0.0 — Java 17+ required

Released Dec 2025. Raises baseline to Java 17, upgrades to Groovy 5, adds Spring 7 + Jackson 3 support. Use v5.5.x for Java 11 compatibility (LTS backport available).

Data-Driven API Tests & Contract / Schema Validation

Data-driven — @ParameterizedTest + REST Assured

```
// Data-driven API test — JUnit 5
@ParameterizedTest
@ParameterizedTest
@CsvSource({
  "1, Alice, 200",
  "999, null, 404",
  "abc, null, 400"
})
void getUser_variousIds(String id,
                        String name,
                        int expectedStatus) {
  ValidatableResponse resp = given()
    .baseUri(BASE_URL)
  .when()
    .get("/users/{id}", id)
  .then()
    .statusCode(expectedStatus);

  if (expectedStatus == 200) {
    resp.body("name", equalTo(name));
  }
}
```

JSON Schema file — user-schema.json

```
// JSON Schema file: src/test/resources/
//                   schemas/user-schema.json
{
  "$schema": "http://json-schema.org/draft-07/schema",
  "type": "object",
  "required": ["id", "name", "email"],
  "properties": {
    "id":    { "type": "integer" },
    "name":  { "type": "string"  },
    "email": { "type": "string",
               "format": "email"  }
  }
}
```

Schema validation test — REST Assured

```
// Assert response matches schema
import static io.restassured.module.jsv
    .JsonSchemaValidator.*;

@Test
void getUser_responseMatchesSchema() {
  given()
    .baseUri(BASE_URL)
  .when()
    .get("/users/1")
  .then()
    .statusCode(200)
    .body(matchesJsonSchemaInClasspath(
      "schemas/user-schema.json"));
}
```

Postman Collections & Newman CLI in CI

Newman CLI — Command Reference

Newman CLI commands

```
# Install Newman globally
npm install -g newman

# Run a collection (basic)
newman run collection.json

# Run with environment file
newman run collection.json \
  -e environments/staging.json

# Generate JUnit XML report (for CI)
newman run collection.json \
  -e environments/staging.json \
  -r cli,junit \
  --reporter-junit-export results/
      newman-junit.xml

# Run specific folder in collection
newman run collection.json \
  --folder "User API Tests"
```

GitHub Actions — Newman in CI Pipeline

.github/workflows/ci.yml — Newman job

```
  api-test:
    needs: unit-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - name: Install Newman
        run: npm install -g newman

      - name: Run API tests
        run: |
          newman run \
            postman/api-tests.json \
            -e postman/env-ci.json \
            -r cli,junit \
            --reporter-junit-export \
            results/newman-junit.xml

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: api-test-results
          path: results/newman-junit.xml
      # upload-artifact stores the file only.
      # Add a test-reporting action (e.g.
      # dorny/test-reporter) to surface results
      # as job summaries or PR check annotations.
```

Integrating REST Assured API Tests into CI

pom.xml — Failsafe for API/integration tests

```
<!-- pom.xml: run API tests via Failsafe -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.5.5</version>
  <executions>
    <execution>
      <goals>
        <goal>integration-test</goal>
        <goal>verify</goal>
      </goals>
    </execution>
  </executions>
  <configuration>
    <groups>api</groups>
    <includes>**/*IT.java</includes>
  </configuration>
</plugin>

<!-- Convention: API test class names end *IT -->
<!-- Run: mvn verify -Dgroups=api           -->
```

GitHub Actions — REST Assured API test job

```
  api-test:
    needs: unit-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-java@v5
        with:
          java-version: '21'
          distribution: temurin
      - uses: actions/cache@v4
        with:
          path: ~/.m2
          key: ${{ runner.os }}-m2-${{
            hashFiles('**/pom.xml') }}

      - name: Run API integration tests
        run: mvn verify -Dgroups=api
        env:
          API_BASE_URL: ${{ vars.STAGING_URL }}
          API_KEY: ${{ secrets.API_KEY }}

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: api-test-results
          path: target/failsafe-reports/
      # upload-artifact stores the XML file.
      # Use a test-reporting action to parse JUnit
      # XML and surface results in the job summary.
```

Analysing API Test Reports & CI Execution Evidence

Reading JUnit XML Reports

target/failsafe-reports/TEST-UserApiIT.xml

```
<!-- JUnit XML — produced by both Failsafe -->
<!-- and Newman -r junit                   -->
<testsuite name="API Tests"
           tests="12" failures="1"
           errors="0"  time="4.32">

  <testcase name="getUser_returns200"
            classname="UserApiIT"
            time="0.45"/>

  <testcase name="createUser_returns201"
            classname="UserApiIT"
            time="0.83">
    <failure message="expected: 201
    but was: 400">
      AssertionError: status code
    </failure>
  </testcase>
</testsuite>
```

API Failure Investigation Checklist

1. Check status code mismatch

Expected 201, got 400? Read the response body in the failure message — the server may return a validation error detail that tells you exactly what's wrong.

2. Verify the test environment config

Was the API_BASE_URL variable pointing at the right environment? Check the CI env-vars. A test that works locally but fails in CI is often a config/environment issue.

3. Check auth token / API key validity

401 Unauthorised in CI often means a secret expired or wasn't injected correctly. Verify the secret name matches between the pipeline YAML and GitHub repo settings.

4. Inspect response time and payload

Timeout errors (408/504) or truncated responses may indicate a downstream service was unavailable during the run. Check if the test is hitting a real service or a mock.

5. Separate contract vs functional failures

Schema validation failure = the response structure changed (contract drift). Status/body assertion failure = functional regression. Triage and route to the right team.

DEMO

Demo: Write → Integrate → Validate → Analyse

01 Writing API Automation Tests with REST Assured

- Add REST Assured 6.0.0 dependency to the course Maven project
- Write a GET test: given baseUri + when get('/users/1') + then statusCode(200)
- Add a JSON Schema validation assertion using matchesJsonSchemaInClasspath()
- Run locally: mvn verify -Dgroups=api — confirm test passes in console

02 Executing API Tests in CI Pipeline

- Add api-test job to .github/workflows/ci.yml with needs: unit-test
- Inject API_BASE_URL via GitHub repository variables (not secrets for non-sensitive URLs)
- Push to feature branch — observe api-test job in Actions tab
- Download failsafe-reports artefact — open the JUnit XML to review results

03 Analysing API Test Reports & Validating Response Data

- Review Newman CLI console output: iteration count, pass/fail, response times
- Demonstrate: deliberately break a schema field name — observe schema validation fail
- Show: 401 failure simulation — remove API key — trace to 'Unauthorised' in log
- Fix and re-run — confirm CI returns green across both REST Assured and Newman jobs

HANDS-ON LAB

Lab: Create → Integrate → Validate

35% Theory | 65% Hands-on

Task 1

Passing API test suite (mvn verify -Dgroups=api)

Create Automated API Tests (REST Assured)

1. Add REST Assured 6.0.0 + json-schema-validator to pom.xml
2. Create UserApiIT.java: write GET /users/{id} test with status + body assertions
3. Add JSON Schema file to src/test/resources/schemas/
4. Add schema validation assertion: matchesJsonSchemaInClasspath("schemas/user-schema.json")

Task 2

Green CI run with api-test job

Integrate API Tests into CI Pipeline

1. Add api-test job to ci.yml: needs: unit-test, run: mvn verify -Dgroups=api
2. Add API_BASE_URL as a GitHub repository variable (Settings → Variables)
3. Push branch and open a PR — confirm api-test job appears and passes
4. Upload failsafe-reports artefact using upload-artifact@v4 with if: always()

Task 3

CI evidence: annotated JUnit XML

Validate Response Data and Status Codes

1. Add a negative test: GET /users/99999 → expect 404 status code
2. Add a POST test: create a user → assert 201 + Location header present
3. Deliberately break one assertion — observe pipeline failure and JUnit XML failure node
4. Fix, re-push, verify CI returns fully green across all three test jobs

Day 5 Deliverables & Key Takeaways

Deliverables You Take Away

- API Automation Test Suite (REST Assured + JSON Schema validator)
- CI Execution Evidence — green pipeline run with api-test job

Key Takeaways

1. API tests catch integration defects faster and more reliably than UI tests — invest in the service layer
2. REST Assured 6.0.0 requires Java 17+; use 5.5.x for Java 11 projects (LTS backport available)
3. JSON Schema validation catches contract drift — the response structure changed, not just one value
4. Newman produces JUnit XML natively (-r junit); upload as an artefact and add a reporting action to surface summaries in CI
5. Inject base URLs and API keys via CI env-vars and secrets — never hardcode in test code

Tomorrow — Day 6: Test Environment, Containers & Test Data · Docker for test execution, env provisioning, test data strategies

Hands-on: Containerise test execution · configure pipeline to use Docker · manage test data across environments

RECAP QUIZ

Module 5 – Quick Knowledge Check

Q1. What is the difference between a status code assertion and a JSON Schema validation in an API test?

- A status code assertion checks the HTTP response code (e.g. 200, 404). A JSON Schema validation checks the shape of the response body — field names, data types, required fields — against a defined schema. Schema validation catches contract drift that status codes alone cannot detect.

Q2. Why is REST Assured 6.0.0 not suitable for Java 11 projects?

- REST Assured 6.0.0 raised the minimum baseline to Java 17 (released Dec 2025). It also upgraded to Groovy 5. Java 11 projects should use REST Assured 5.5.x, which has a backported LTS release with Spring 7 MockMvc support.

Q3. How does Newman produce CI-compatible test results, and how do they surface in GitHub Actions?

- Newman with -r junit and --reporter-junit-export produces a JUnit XML file. upload-artifact@v4 (with if: always()) stores the file so it can be downloaded from the pipeline run. To surface results as job-summary annotations or PR check results, teams add a dedicated test-reporting action (e.g. dorny/test-reporter) that parses the JUnit XML — upload-artifact alone does not do this.

Q4. A REST Assured test passes locally but gets a 401 in CI. What are the two most likely causes?

- 1) The API key secret was not correctly injected — check that the secret name in the YAML matches the GitHub repo secret name exactly. 2) The API key has expired or the secret value is wrong. Check the CI env-var injection with echo (masked) and verify the key is still valid.